CS 422 Assignment 3: TCP Congestion Control Implementation

Option 1: Gentle Reno as an eBPF `struct_ops` congestion control

Daniel Chindris Saverio Miller Richie Tan Philip Wisniewski
Emma Zheng

CS 422: Computer Networks, Fall 2026
Repository: github.com/philwisniewski/cs-422-assignment-3

Abstract

We implemented the congestion control algorithm from Assignment 2, Question 3(b), "Gentle Reno", as an eBPF `struct_ops` program that registers a kernel `tcp_congestion_ops` named `gentle_reno`. Our Assignment 2 Python iperf3 client selects it on the data socket with `setsockopt(TCP_CONGESTION)` and reads back the algorithm actually used. We compared it with CUBIC, Reno and BBR on 10 public iperf3 servers, using 5 repetitions of 10 s tests sampled every 0.2 s (Linux 6.17.0-1022-nvidia). Gentle Reno had the lowest RTT inflation (4.46 ms vs. 7.29 ms for CUBIC and 14.84 ms for BBR) and almost no retransmissions (0.01 % of bytes). It also had much lower throughput (mean 25.52 Mbit/s vs. 132.17, 104.02 and 171.19 Mbit/s). The cause is that its slow-start exit fires early, after which its +0.5 packet/RTT growth cannot fill a long-RTT path within 10 s.

Contents

1 Algorithm (Assignment 2, Q3(b)) 3

2 Implementation 4

2.1 Division of work between the kernel and the program 4
2.2 How the congestion window is updated 4
2.3 Initial state and parameters 5
2.4 Changes from the Assignment 2 pseudocode 5

3 Selecting the algorithm on the data socket 6

4 Experimental setup 7

5 Results 8

5.1 Time series and scatter plots (A2 Q2(b), all four algorithms) 8
5.2 Aggregate throughput and RTT statistics 9
5.3 What the eBPF program saw 11

6 Discussion of the observed differences **11**

7 Reproduction **13**

7.1 Kernel and required features 13

7.2 Build, load and check 13

7.3 Run the experiments and build this report 13

7.4 Raw data and configuration 14

7.5 Where the code is 14

8 Use of AI tools **15**

A Per-algorithm plots at the representative destination **15**

B Test order **22**

1 Algorithm (Assignment 2, Q3(b))

Gentle Reno keeps Reno's loss-driven AIMD structure but makes both halves gentler, and adds RTT- and throughput-based rules for leaving slow start without a loss:

- **Loss:** multiplicative decrease by $\beta = 0.7$ instead of 0.5, at most once per loss episode.
- **Growth:** additive increase of $a = 0.5$ packet per RTT instead of 1.
- **RTT:** leave slow start (without a cut) once the RTT rises more than $D = \max(4\,\text{ms}, \text{min_rtt}/8)$ above the minimum, which means a queue is forming.
- **Throughput:** leave slow start once the delivery rate has grown by less than 25 % for three rounds in a row, which means the pipe is full.

The original pseudocode (docs/a2_pseudocode.txt) is:

```
# Custom TCP Algorithm

General idea: gentler Reno

Parameters:  β = 0.7            multiplicative-decrease factor
             a = 0.5            additive increase, in packets per RTT
             D = max(4 ms, min_rtt/8)    RTT-rise threshold for leaving slow start
State:       cwnd = 10, ssthresh = ∞, min_rtt = ∞, full_bw = 0, flat_rounds = 0

On every ACK:
    min_rtt ← min(min_rtt, latest RTT sample)
    if in slow start (cwnd < ssthresh):
        cwnd ← cwnd + 1                          # doubles each RTT
    else:
        cwnd ← cwnd + a / cwnd                   # +a packets per RTT

Once per RTT, during slow start only:
    bw ← bytes delivered in the last RTT / RTT       # measured throughput
    if bw ≥ 1.25 · full_bw: full_bw ← bw; flat_rounds ← 0
    else:                   flat_rounds ← flat_rounds + 1
    if (srtt − min_rtt > D) or (flat_rounds ≥ 3):   # queue is building or the pipe is
     full
        ssthresh ← cwnd                          # leave slow start without cutting

On 3 duplicate ACKs / SACK loss (fast retransmit):
    ssthresh ← max(β · cwnd, 2)
    cwnd ← ssthresh                              # at most once per RTT (one cut per
    loss event)
    fast recovery as in Reno / NewReno

On retransmission timeout (RTO):
    ssthresh ← max(β · cwnd, 2)
    cwnd ← 1                                     # severe congestion: restart with slow
    start
    RTO ← 2 · RTO (exponential backoff); RTO is computed from srtt and rttvar
```

2 Implementation

The program is bpf/gentle_reno.bpf.c. It is compiled with clang to a BPF object and registered with `bpftool struct_ops register`, after which `gentle_reno` appears in `/proc/sys/net/ipv4/tcp_available_congestion_control` next to `cubic`, `reno` and `bbr`. The struct that registers the hooks is at bpf/gentle_reno.bpf.c:228-236.

2.1 Division of work between the kernel and the program

The Linux TCP stack still does everything except the window policy: ACK processing, loss detection (dup ACKs, SACK, RACK, RTO timers), fast recovery with PRR (RFC 6937), retransmission, and RTT estimation (srtt, rttvar, RTO). It calls our hooks at each decision point:

Hook	Called when	What Gentle Reno does	Signal
`gr_init`	connection established	reset per-connection state, start the first round	–
`gr_ssthresh`	once per loss episode (enter Recovery after 3 dup ACKs/SACK/RACK, or Loss after RTO)	return $\max(\lfloor 7\,\text{cwnd}/10 \rfloor, 2)$	loss
`gr_cong_avoid`	each ACK that advances `snd_una`, outside recovery	slow start: +1 per ACKed packet; else +1 packet per $2 \cdot \text{cwnd}$ ACKed ($= +0.5$/RTT)	RTT (ACK clock)
`gr_pkts_acked`	each ACK, with its RTT sample	track min RTT; at each round end evaluate the slow-start exit rules	RTT, throughput
`gr_set_state`	CA state change	after an RTO re-arm the slow-start exit	loss (timeout)
`gr_undo_cwnd`	kernel proves a loss spurious (DSACK, timestamps)	restore $\max(\text{cwnd}, \text{prior_cwnd})$, like Reno	loss

2.2 How the congestion window is updated

Loss (bpf/gentle_reno.bpf.c:121-129). `gr_ssthresh` returns $0.7 \cdot \text{cwnd}$ (at least 2). In fast recovery, PRR moves cwnd smoothly down to that value, so after recovery cwnd = ssthresh = $0.7 \cdot \text{cwnd}_{\text{before}}$. Because the kernel calls `ssthresh` only when it *enters* recovery and not once per lost packet, the cut happens at most once per loss episode (about once per RTT). On an RTO the kernel sets cwnd = 1 after calling `ssthresh`, so slow start restarts toward $0.7 \cdot \text{cwnd}_{\text{before}}$, as in the pseudocode. The loss signals we log are retransmitted segments per interval (Δ`tcpi_total_retrans`), RTO count (`tcpi_total_rto`) and `ca_state` (3 = Recovery after dup ACKs, 4 = Loss after an RTO). On a shallow-buffer test path we measured `ssthresh`/cwnd-before-loss at 0.70–0.73 for `gentle_reno`, against 0.50–0.55 for `reno` and about 0.7 for `cubic`.

Growth and RTT (bpf/gentle_reno.bpf.c:132-147). Growth happens per round trip. `tcp_cong_avoid_ai(tp, 2*cwnd, acked)` adds one packet after $2 \cdot \text{cwnd}$ packets are ACKed. A full window of ACKs arrives once per RTT, so this is +0.5 packet/RTT. The window grows only while the sender is cwnd-limited (`tcp_is_cwnd_limited` logic), as in every kernel algorithm.

RTT and throughput in slow start (bpf/gentle_reno.bpf.c:150-202). On every ACK the program tracks `min_rtt_us` (connection minimum) and `round_min_rtt_us` (minimum within the current round). A *round* starts by recording end_seq = `snd_nxt` and ends when `snd_una` passes it, i.e. one RTT. This is the round definition that CUBIC's HyStart uses. At the end of each slow-start round:

- **Throughput:** bw = (`delivered` − $\texttt{delivered}_{\text{round start}}$) · $10^6/\Delta t_{\mu s}$ in packets/s. If 4 bw ≥ 5 full_bw, then full_bw ← bw and the flat counter resets. Otherwise `flat_rounds` is incremented. Slow start ends after 3 flat rounds.
- **RTT:** slow start ends if round_min_rtt > min_rtt + max(4 ms, min_rtt/8).

Leaving slow start sets ssthresh = cwnd with no cut, so the window reached is kept and growth continues at +0.5/RTT.

2.3 Initial state and parameters

Parameters are `#define`s at bpf/gentle_reno.bpf.c:27-38; per-connection state (32 bytes in `icsk_ca_priv`, within the kernel's 104-byte limit) is at bpf/gentle_reno.bpf.c:41-50; initialisation is bpf/gentle_reno.bpf.c:102-116. All arithmetic is integer because BPF has no floating point.

Quantity	Value	Where
cwnd	10 packets (kernel IW10)	kernel default
ssthresh	∞ (`TCP_INFINITE_SSTHRESH`): start in slow start	kernel default
min_rtt, round_min_rtt	∞ (`0xffffffff`)	`gr_init`
full_bw, flat_rounds, ss_done	0	`gr_init`
round	`snd_nxt`, `tcp_mstamp`, `delivered` at init	`start_round`
β	7/10	`BETA_NUM/BETA_DEN`
a	1/2 packet per RTT	`AI_DIVISOR = 2`
Minimum cwnd after a cut	2	`MIN_CWND`
D	max(4 ms, min_rtt/8)	`SS_DELAY_MIN_US`, `SS_DELAY_SHIFT`
Throughput plateau	< 1.25× growth for 3 rounds	`SS_BW_GROWTH_*`, `SS_FLAT_ROUNDS`

The program also keeps global counters (`stat_init`, `stat_loss_events`, `stat_ss_exit_delay`, `stat_ss_exit_bw`, `stat_undo`). They are read with `sudo make -C bpf stats` and snapshotted before and after each experiment into `metadata.json` (run_experiments.py:79-102).

2.4 Changes from the Assignment 2 pseudocode

1. **The RTT rule uses the minimum RTT sample in the round, not srtt.** srtt is a smoothed average that lags several RTTs behind and is inflated by delayed ACKs and jitter. The per-round minimum reacts within one round and ignores single noisy samples. Linux HyStart makes the same choice. D is unchanged.

2. **The slow-start exit is evaluated at most once per slow-start phase** (`ss_done`) and is re-armed after an RTO (`gr_set_state`, bpf/gentle_reno.bpf.c:205-215). The pseudocode left this implicit.

3. **Cut and recovery mechanics come from the kernel.** `ssthresh()` supplies only the target, and PRR moves cwnd to it during recovery. The end state matches the pseudocode's "cwnd ← ssthresh".

4. **Undo of spurious cuts.** The kernel requires an `undo_cwnd` hook (bpf/gentle_reno.bpf.c:218-225). It restores the pre-loss window when a retransmission turns out to be unnecessary, as Reno does.

5. **Growth only when cwnd-limited.** This prevents window inflation when the application or the receive window is the real limit.

6. **Fixed-point parameters:** $\beta = 7/10$ in integers, and $a = 0.5$ as "+1 per 2·cwnd ACKs".

3 Selecting the algorithm on the data socket

The Assignment 2 client (iperf3_client.py) sets the algorithm on the *data* socket before `connect()`, so the whole connection, including slow start, uses it (iperf3_client.py:164-191, which calls tcp_info.py:138-140):

```
sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_CONGESTION, b"gentle_reno")
sock.connect(addr)
actual = sock.getsockopt(socket.IPPROTO_TCP, socket.TCP_CONGESTION, 16)
```

After the connection is up the client reads the algorithm back (tcp_info.py:132-135) and stores it as `algorithm_actual` in every sample row and every table. In every run of this experiment the requested and actual algorithm matched (the "actual" column of Table 1 and the panel titles of the figures). Before any test, `preflight` (run_experiments.py:105-126) opens a socket for each algorithm and records what the kernel reports: `cubic`→`cubic`, `reno`→`reno`, `bbr`→`bbr`, `gentle_reno`→`gentle_reno`.

4 Experimental setup

Host	`Linux jensen 6.17.0-1022-nvidia #22-Ubuntu SMP PREEMPT_DYNAMIC Wed May 27 19:31:58 UTC 2026 aarch64 aarch64`
Kernel	`6.17.0-1022-nvidia` (aarch64, Ubuntu)
Available algorithms	`reno cubic bbr gentle_reno` (default `cubic`)
Python	3.12.3
Command	`python3 run_experiments.py -servers servers.csv -n 10 -t 10 -r 5 -seed 42`
Run directory	`results/20261018-133538` (started 2026-10-18 13:35:38 -0400, finished 2026-10-18 14:22:02 -0400)
Destinations	10, drawn at random (seed 42) from 189 servers in `servers.csv`; candidates that failed a 3 s probe were replaced (15 attempts, `run_experiments.py:150-198`)
Duration / sampling	10 s per test, sampled every 0.2 s, 2 s pause between tests
Repetitions	5 of every destination × algorithm: 200 scheduled tests, 190 completed
Failed tests	10, all at `speed1.fiberby.dk:9212` (`connect_error`: the server refused the connection). They are reported, not retried into the statistics.

Destinations. All four algorithms used the same 10 destinations:

#	Destination	Location	Provider	Gbit/s
1	`41.110.39.130:5201`	Algiers, DZ	DATAPACKET	10
2	`akl.linetest.nz:5301`	Auckland, NZ	2Degrees	10
3	`lon.speedtest.clouvider.net:5207`	London, GB	Cloudvider	10
4	`speedtest.sin1.sg.leaseweb.net:5207`	Singapore, SG	LeaseWeb	10
5	`speedtest.wdc2.us.leaseweb.net:5203`	Washington, US	LeaseWeb	10
6	`72.1.86.46:30006`	Auckland, NZ	FortiSASE	10
7	`speed1.fiberby.dk:9212`	Copenhagen, DK	Fiberby	25
8	`41.210.185.162:5201`	Kampala, UG	DATAPACKET	2x10
9	`speedtest.nyc1.us.leaseweb.net:5207`	New York City, US	LeaseWeb	10
10	`speedtest.phx1.us.leaseweb.net:5206`	Phoenix, US	LeaseWeb	10

Test order. For each repetition, for each destination, the four algorithms run back to back. Their order is rotated by (rep + destination index) mod 4, so no algorithm always goes first and slow drift in path conditions is spread across algorithms (`run_experiments.py:213-267`). The full order is listed in Appendix B and in `test_order.csv`.

Logged per sample (`iperf3_client.py:275-332`; `samples/*.csv`): goodput (Δ`tcpi_bytes_acked`/Δt), `snd_cwnd`, `snd_ssthresh`, srtt, rttvar, min_rtt, RTO, kernel delivery rate, and the loss signals: Δ`tcpi_total_retrans` per interval, `bytes_retrans`, `lost`, `total_rto` and `ca_state`.

Representative destination. `lon.speedtest.clouvider.net:5207`. The analysis picks the destination with complete data for every algorithm whose median goodput is closest to the median over all destinations (`analysis.py:314-352`). All four algorithms are plotted for this destination with identical axis limits.

5 Results

5.1 Time series and scatter plots (A2 Q2(b), all four algorithms)

Figures 1 and 2 repeat the Assignment 2 Q2(b) plots for all four algorithms on the representative destination, side by side with shared axes. The individual per-algorithm versions are in Appendix A. They are generated by `make_plots` (`analysis.py:577-687`), using `plot_timeseries` (`analysis.py:435-446`) and `plot_scatter` (`analysis.py:471-486`).

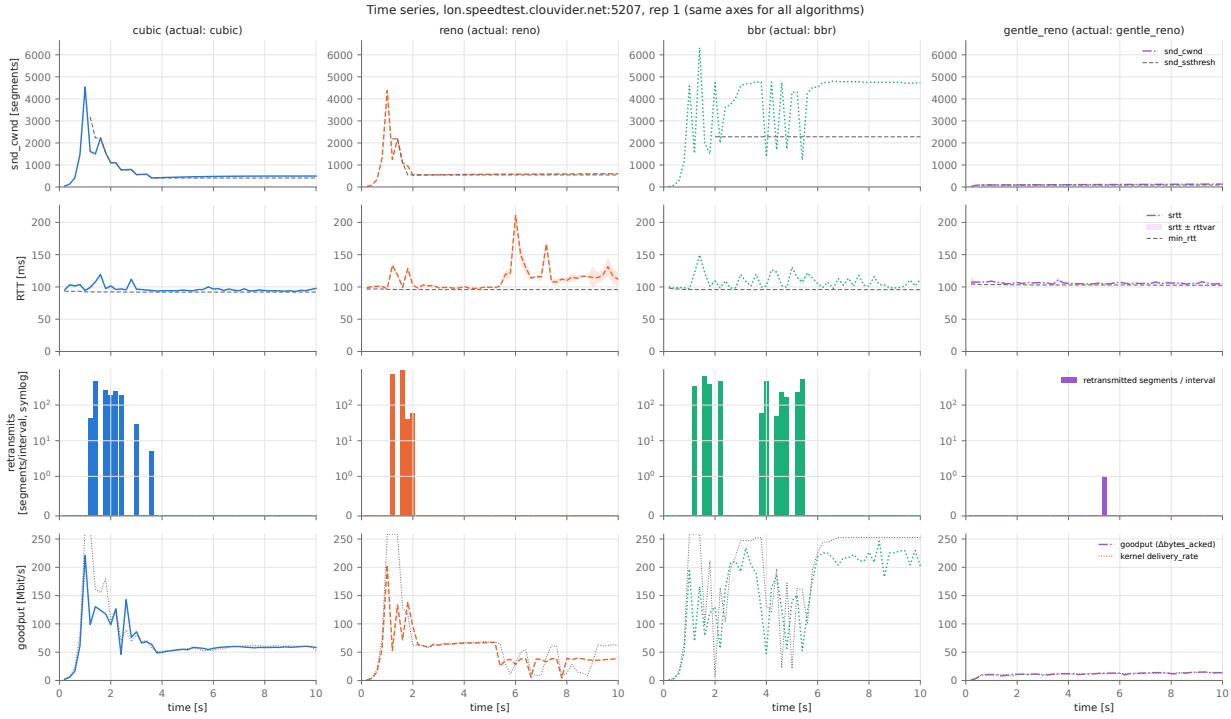


Figure 1: Time series at `lon.speedtest.clouvider.net:5207`, repetition 1: congestion window and ssthresh, RTT (srtt, srtt±rttvar, min_rtt), retransmitted segments per interval (loss signal, symlog), and goodput. Columns are CUBIC, Reno, BBR and Gentle Reno, each titled with the requested and actual algorithm; every row uses the same y-axis across algorithms. File: `plots/q3_timeseries_compare.pdf`.

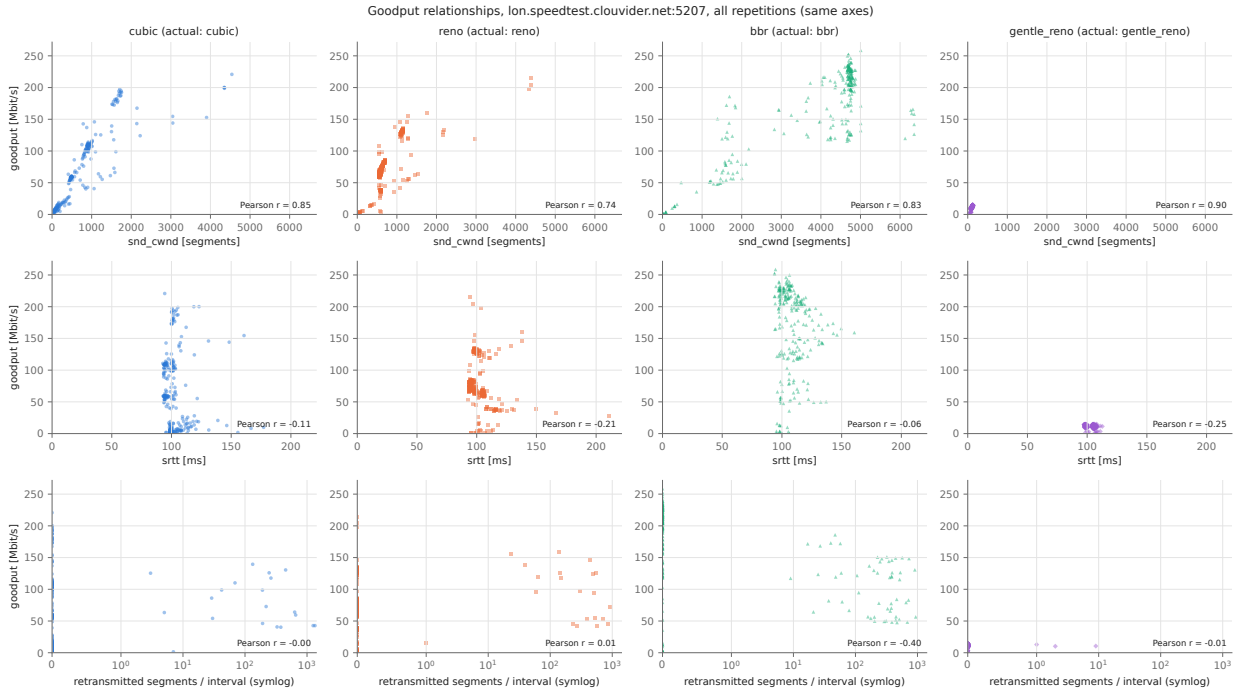


Figure 2: Goodput against cwnd, srtt and retransmissions per interval at `lon.speedtest.clouvider.net:5207`, all repetitions, with the Pearson correlation in each panel. Same axes for all algorithms. File: `plots/q3_scatter_compare.pdf`.

5.2 Aggregate throughput and RTT statistics

Table 1 pools every 0.2 s interval sample of every completed run (all destinations and repetitions). Percentiles use `numpy.percentile` with its default linear interpolation between order statistics (Hyndman–Fan type 7). RTT inflation, retransmission rate and RTO count are computed per run and then averaged. The code is `build_tables` (`analysis.py:203-294`) and `run_stats` (`analysis.py:163-200`).

Statistic	CUBIC	Reno	BBR	Gentle Reno
Runs / interval samples	48 / 2400	47 / 2350	48 / 2400	47 / 2350
Goodput mean [Mbit/s]	132.2	104.0	171.2	25.5
Goodput median [Mbit/s]	76.7	53.3	108.4	10.1
Goodput p5 [Mbit/s]	0.12	0.12	0.12	1.27
Goodput p95 [Mbit/s]	394.4	322.7	633.6	74.8
srtt mean [ms]	137.0	139.2	146.5	135.6
srtt median [ms]	130.6	132.9	135.8	130.6
srtt p95 [ms]	250.8	253.1	270.1	247.8
RTT inflation (srtt − min_rtt) [ms]	7.29	7.63	14.84	4.46
Retransmitted bytes [%]	1.63	1.86	5.09	0.01
Retransmitted segments / run	1120	1044	7760	17
RTOs / run	0.02	0.00	0.04	0.00

Table 1: Aggregate statistics per algorithm. Source: `tables/aggregate_by_algorithm.csv`. The "actual" algorithm recorded on every socket was `cubic`, `reno`, `bbr`, `gentle_reno` respectively.

Tables 2 and 3 break the results down by destination. Each cell is the mean ± standard deviation over repetitions of the per-run mean. Bold marks the best algorithm on that path, and a superscript gives the number of completed runs when it is not 5.

Destination	**CUBIC**	**Reno**	**BBR**	**Gentle Reno**
`41.110.39.130:5201`	92.5 ± 10.2	53.7 ± 19.4	**95.6 ± 9.2**	13.0 ± 10.6
`akl.linetest.nz:5301`	67.0 ± 7.3	36.8 ± 1.0	**72.9 ± 10.2**	9.4 ± 9.0
`lon.speedtest.clouvider.net:5207`	65.5 ± 46.7	77.5 ± 22.7	**164.0 ± 13.9**	11.1 ± 0.8
`speedtest.sin1.sg.leaseweb.net:5207`	28.2 ± 13.4	34.2 ± 10.4	**56.5 ± 7.7**	3.9 ± 2.3
`speedtest.wdc2.us.leaseweb.net:5203`	320.8 ± 84.9	234.3 ± 15.7	**414.3 ± 83.9**	53.0 ± 11.1
`72.1.86.46:30006`	48.4 ± 4.7	37.1 ± 1.2	**74.3 ± 1.0**	8.1 ± 4.8
`speed1.fiberby.dk:9212`	6.6 ± 0.3$^{(3)}$	6.3 ± 0.3$^{(2)}$	6.4 ± 0.1$^{(3)}$	**22.3 ± 13.4**$^{(2)}$
`41.210.185.162:5201`	44.6 ± 5.9	35.5 ± 9.3	**60.1 ± 3.8**	3.7 ± 0.6
`speedtest.nyc1.us.leaseweb.net:5207`	373.9 ± 68.8	275.4 ± 44.2	**445.1 ± 192.4**	100.1 ± 97.6
`speedtest.phx1.us.leaseweb.net:5206`	224.2 ± 49.7	190.8 ± 23.3	**256.7 ± 52.5**	28.6 ± 3.7
All destinations	132.2 ± 131.0	104.0 ± 94.1	**171.2 ± 163.1**	25.5 ± 44.0

Table 2: Mean goodput per destination [Mbit/s]. Source: `tables/algorithm_comparison.csv`.

Destination	**CUBIC**	**Reno**	**BBR**	**Gentle Reno**
`41.110.39.130:5201`	136.0 ± 5.1	148.0 ± 21.7	144.4 ± 3.6	**132.3 ± 1.7**
`akl.linetest.nz:5301`	190.1 ± 2.3	190.1 ± 1.6	199.9 ± 6.2	**188.4 ± 2.0**
`lon.speedtest.clouvider.net:5207`	102.8 ± 6.4	**102.7 ± 6.3**	108.4 ± 3.6	103.1 ± 3.4
`speedtest.sin1.sg.leaseweb.net:5207`	241.8 ± 6.9	239.7 ± 10.3	260.8 ± 12.8	**238.5 ± 10.6**
`speedtest.wdc2.us.leaseweb.net:5203`	35.1 ± 1.8	**33.0 ± 1.9**	45.1 ± 6.9	37.2 ± 3.1
`72.1.86.46:30006`	200.7 ± 5.6	203.8 ± 4.7	209.3 ± 3.9	**196.9 ± 6.2**
`speed1.fiberby.dk:9212`	121.2 ± 5.3$^{(3)}$	118.4 ± 0.2$^{(2)}$	127.7 ± 10.2$^{(3)}$	**113.1 ± 1.2**$^{(2)}$
`41.210.185.162:5201`	251.6 ± 2.8	254.5 ± 5.4	256.0 ± 3.4	**246.1 ± 2.1**
`speedtest.nyc1.us.leaseweb.net:5207`	**30.3 ± 1.9**	30.8 ± 1.4	41.3 ± 17.1	32.1 ± 2.2
`speedtest.phx1.us.leaseweb.net:5206`	**54.2 ± 2.9**	58.2 ± 2.6	64.4 ± 3.9	54.8 ± 1.8
All destinations	137.0 ± 79.8	139.2 ± 81.2	146.5 ± 80.5	**135.6 ± 78.4**

Table 3: Mean srtt per destination [ms]; bold = lowest. Source: `tables/algorithm_comparison.csv`.

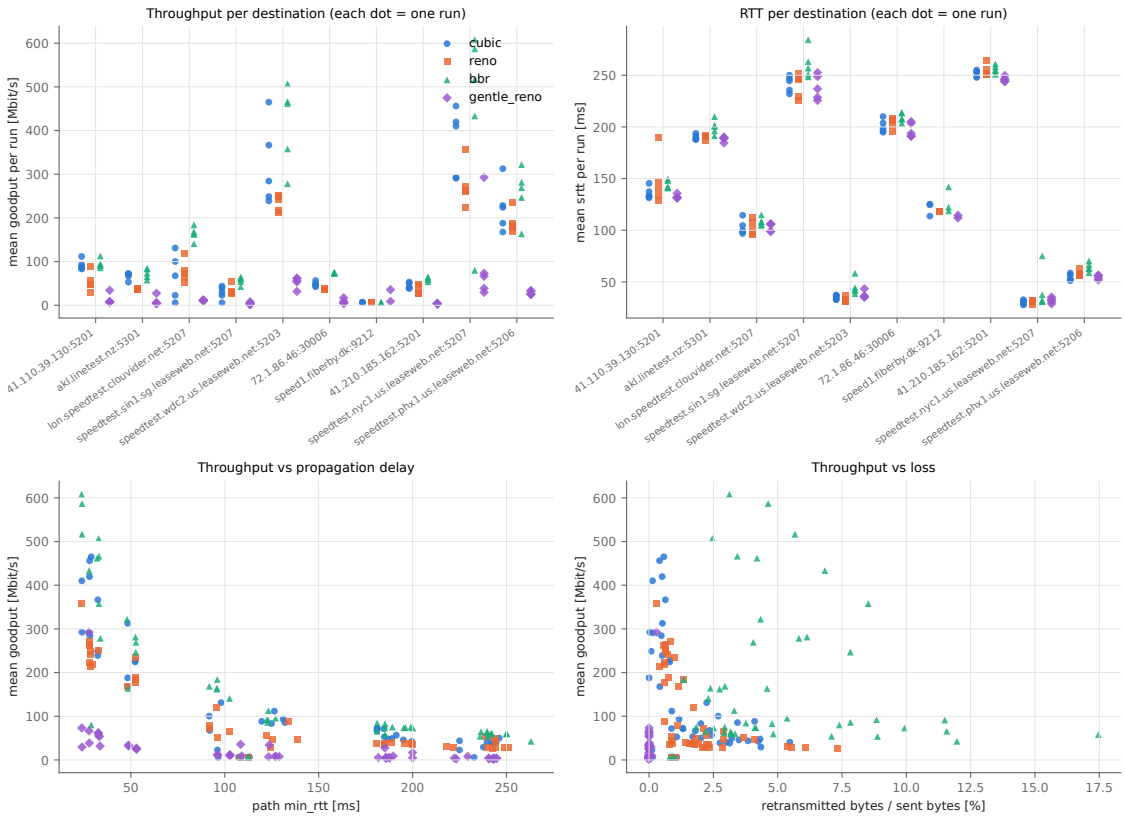


Figure 3: Per-run mean goodput and srtt for each destination (top), and goodput against the path's min RTT and against the retransmission rate (bottom). One marker per run. Generated by `plot_regimes` (`analysis.py:536-574`). File: `plots/q3_regimes.pdf`.

5.3 What the eBPF program saw

Counters read from the program after the experiment: 47 connections used `gentle_reno`. Slow start ended through the throughput-plateau rule 44 times and through the RTT-rise rule 2 times. There were 15 loss episodes (`ssthresh` calls) and 5 spurious-loss undos. The mean congestion window was 149 segments for Gentle Reno, compared with 1071 (CUBIC), 832 (Reno) and 3289 (BBR). As a fraction of each path's BDP (the largest delivery-rate × min-RTT estimate any algorithm reached on that destination), the mean window was 7 % for Gentle Reno, 47 % for CUBIC, 37 % for Reno and 146 % for BBR.

6 Discussion of the observed differences

BBR: highest throughput, most queueing and loss. BBR had the highest mean goodput on *every* destination except the anomalous `speed1.fiberby.dk` (Table 2): 171.19 Mbit/s overall. It also had the highest RTT inflation (14.84 ms) and by far the most retransmissions (5.09 % of bytes, 7759.90 segments per run). BBR (v1 in this kernel) does not treat loss as a congestion signal. It paces at its bottleneck-bandwidth estimate, probes with 1.25× gain, and caps cwnd at about 2× the estimated BDP. Its window sat above the path BDP (146 %), so it keeps a standing queue and overflows shallow buffers. Its throughput does not collapse after loss the way the loss-based

algorithms' does. Figure 1 shows this: BBR's cwnd stays near 4700 segments despite repeated retransmission bursts, while CUBIC and Reno drop to a few hundred segments after their first losses around $t \approx 1$–2 s.

CUBIC vs. Reno. Both are loss-based and saw similar RTT (137.02 vs. 139.16 ms) and retransmission rates. CUBIC was faster on average (132.17 vs. 104.02 Mbit/s). It cuts by only 30 % (Reno halves), and its cubic growth function depends on time since the last loss, not on RTT. On the long-RTT paths here (most min RTTs are 100–250 ms), Reno's +1 packet/RTT takes many seconds to regain the window after a cut. CUBIC returns to its previous maximum in a few seconds. The gap is largest on the low-RTT, high-bandwidth US paths (`wdc2`, `nyc1`), where the window can grow the most within 10 s.

Gentle Reno: lowest delay and loss, lowest throughput. Gentle Reno had the lowest mean srtt (135.61 ms), the lowest RTT inflation (4.46 ms) and essentially no retransmissions (16.55 segments per run, against more than 1000 for the others). Its mean goodput was only 25.52 Mbit/s. The data point to one cause:

- The slow-start exit fired in nearly every connection, almost always through the throughput-plateau rule (44 of 47), at a window of roughly 50–200 segments in most runs. That is far below the path BDP: the mean window was only 7 % of BDP. In Figure 1 the Gentle Reno cwnd flattens within the first second and never approaches the 4000+ segments the other algorithms reach.
- After the exit, growth is +0.5 packet/RTT. At 100 ms RTT that is only about +50 packets in a 10 s test, so the window never reaches the buffer and almost never causes a loss. This explains the low RTT and loss, and also the low throughput. The scatter plot (Figure 2) shows a high goodput–cwnd correlation for Gentle Reno: it is window-limited by its own cwnd, not by the network.
- The one run whose slow start did not exit early (`nyc1`, repetition 1) reached 292 Mbit/s, matching CUBIC on that path. The control law can use the bandwidth once the window is large.

The likely reason the plateau rule fires early is that some early rounds are not cwnd-limited. During the first RTTs the receiver's advertised window (still being autotuned up) or ACK aggregation can hold delivery flat for a few rounds. The rule then counts those rounds as "pipe full". BBR's full-bandwidth detector avoids this by ignoring application- or receiver-limited rounds. Adding the same check (skip rounds in which the flow was not cwnd-limited) and requiring the throughput plateau to agree with a modest RTT rise would make the exit much less premature. The gentle $a = 0.5$ increase also makes recovery from an early exit very slow on long-RTT paths. A HyStart-like fallback, or a CUBIC-style growth curve after an early exit, would address that. The algorithm as designed in Assignment 2 is very conservative. It almost never builds a queue, but in 10 s tests over 30–250 ms paths it leaves most of the bandwidth unused. Over a much longer transfer its throughput would keep rising toward the path capacity.

The `speed1.fiberby.dk` anomaly. On this path every loss-based and BBR run had a median of 0.12 Mbit/s with a short burst, and half of the scheduled tests failed (10 failures, all on this server).

This points to server-side throttling. Gentle Reno looks "best" there only because 2 of its runs completed with a short continuous transfer. We do not draw conclusions from this destination.

7 Reproduction

7.1 Kernel and required features

- Kernel used for all results: `6.17.0-1022-nvidia` (Ubuntu, aarch64); exact `uname -a` in Section 4 and in `metadata.json`. Any Linux $\geq$ 5.13 with BTF works.
- Kernel configuration:
  `CONFIG_BPF_SYSCALL=y`, `CONFIG_BPF_JIT=y`
  `CONFIG_DEBUG_INFO_BTF=y` (`/sys/kernel/btf/vmlinux` must exist)
  `CONFIG_TCP_CONG_ADVANCED=y`, and `CONFIG_TCP_CONG_BBR` for the BBR comparison
  The program uses the BPF `tcp_congestion_ops struct_ops` and the `tcp_slow_start` / `tcp_cong_avoid_ai` kfuncs.
- Tools: clang $\geq$ 12, libbpf headers (`libbpf-dev`), bpftool (`linux-tools-$(uname -r)` or `linux-tools-generic`), Python 3 with numpy and matplotlib.
- Root (or `CAP_BPF` + `CAP_NET_ADMIN`) to register the program and select `bbr`/`gentle_reno`.

7.2 Build, load and check

```
sudo apt install clang llvm libbpf-dev linux-tools-common linux-tools-$(uname -r) \
                 python3-numpy python3-matplotlib
make -C bpf              # dump vmlinux.h from the running kernel, clang -target bpf
sudo make -C bpf load    # bpftool struct_ops register gentle_reno.bpf.o
cat /proc/sys/net/ipv4/tcp_available_congestion_control  # ... gentle_reno
sudo make -C bpf status
sudo make -C bpf stats  # program counters
python3 -c "import socket; s=socket.socket(); \
  s.setsockopt(6, 13, b'gentle_reno'); print(s.getsockopt(6, 13, 16))"
sudo make -C bpf unload
```

See bpf/Makefile.

7.3 Run the experiments and build this report

```
# build + load + all tests + plots
sudo ./run_all.sh -n 10 -t 10 -i 0.2 -r 5 --seed 42
# re-make plots and tables only
python3 run_experiments.py --plot-only results/<run>
# this PDF -> report/report.pdf (default: newest results/<run>)
python3 report/make_report.py [results/<run>]
```

run_all.sh builds and registers the program, runs run_experiments.py for all four algorithms, and prints the program counters. report/make_report.py (report/make_report.py:337-376) reads `metadata.json`, `test_order.csv`, `destinations.csv` and `tables/*.csv` from the run directory

and writes the tables and numbers in this report into `report/generated/`. It finds the line ranges of the linked functions in the source, then runs `pdflatex` on `report/report.tex`. Building the report needs only Python 3 and a TeX distribution with `booktabs`, `listings`, `ulem` and `hyperref`. A local network for testing without public servers is in `tools/local_testbed.sh`. The run `results/20261018-133227` is a sanity check on that local network and is not used in this report.

7.4 Raw data and configuration

Everything is in `results/20261018-133538/`:

File	Content
`samples/r<rep>_d<dest>_<alg>.csv`	raw per-interval samples (all `TCP_INFO` fields listed above)
`runs.jsonl`	one record per test: requested and actual algorithm, status, server results, errors
`metadata.json`	kernel, available/allowed algorithms, arguments, seed, `TCP_INFO` layout, eBPF registration and counters before and after
`test_order.csv`	exact execution order, attempts and status
`destinations.csv`, `selection_log.csv`	chosen destinations and the selection probes
`tables/*.{csv,md}`	per-run, per-destination and aggregate tables
`plots/*.pdf`	every figure in this report

Dependencies are in `requirements.txt` (numpy, matplotlib). The server list is `servers.csv` (from iperf3serverlist.net, parsed by `servers.py`).

7.5 Where the code is

What	Code
Parameters / per-connection state	`bpf/gentle_reno.bpf.c:27-38`, `bpf/gentle_reno.bpf.c:41-50`
Initial state	`bpf/gentle_reno.bpf.c:102-116`
Loss response ($\beta = 0.7$)	`bpf/gentle_reno.bpf.c:121-129`
Additive increase / slow start	`bpf/gentle_reno.bpf.c:132-147`
RTT and throughput slow-start exit	`bpf/gentle_reno.bpf.c:150-202`
RTO re-arm / spurious-loss undo	`bpf/gentle_reno.bpf.c:205-215`, `bpf/gentle_reno.bpf.c:218-225`
Registration (`tcp_congestion_ops`)	`bpf/gentle_reno.bpf.c:228-236`
Select algorithm on data socket	`iperf3_client.py:164-191`, `tcp_info.py:138-140`
Record actual algorithm	`tcp_info.py:132-135`, `iperf3_client.py:447-572`
Sampling (cwnd, RTT, goodput, loss)	`iperf3_client.py:275-332`
Experiment loop and test order	`run_experiments.py:213-267`
eBPF counter snapshot	`run_experiments.py:79-102`
Tables and aggregate statistics	`analysis.py:203-294`, `analysis.py:163-200`
Representative destination	`analysis.py:314-352`
Figures 1, 2, 3 and Appendix A	`analysis.py:577-687`, `analysis.py:435-446`, `analysis.py:471-486`, `analysis.py:536-574`
This report	`report/make_report.py:337-376`

8 Use of AI tools

AI tools (Claude, via Claude Code) were used to help brainstorm the design, draft the eBPF program, the experiment and analysis scripts, the local test network, the documentation, and the generator for this report. We reviewed the generated code, built and ran it on our machine, and checked the behaviour of the algorithm on live connections (loss cut ratio, slow-start exit counters, and the algorithm reported by the socket).

A Per-algorithm plots at the representative destination

These are the Assignment 2 Q2(b) figures made separately for each algorithm, at `lon.speedtest.clouvider.net:5207`, with the same axis limits for all four algorithms.

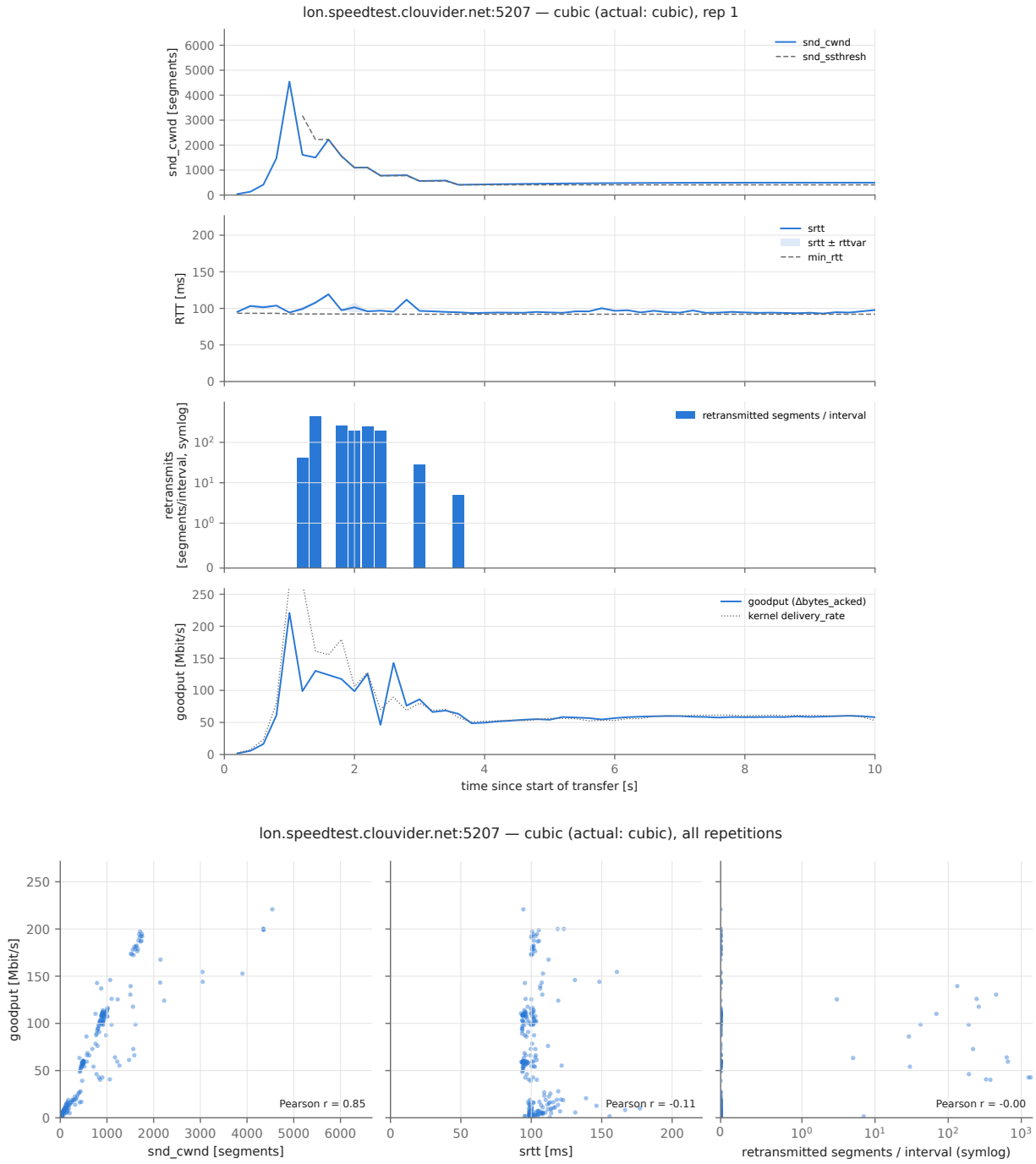


Figure 4: CUBIC (`cubic`) at `lon.speedtest.clouvider.net:5207`: time series (top, repetition 1) and scatter (bottom, all repetitions). Files: `plots/q2_timeseries_cubic.pdf`, `plots/q2_scatter_cubic.pdf`.

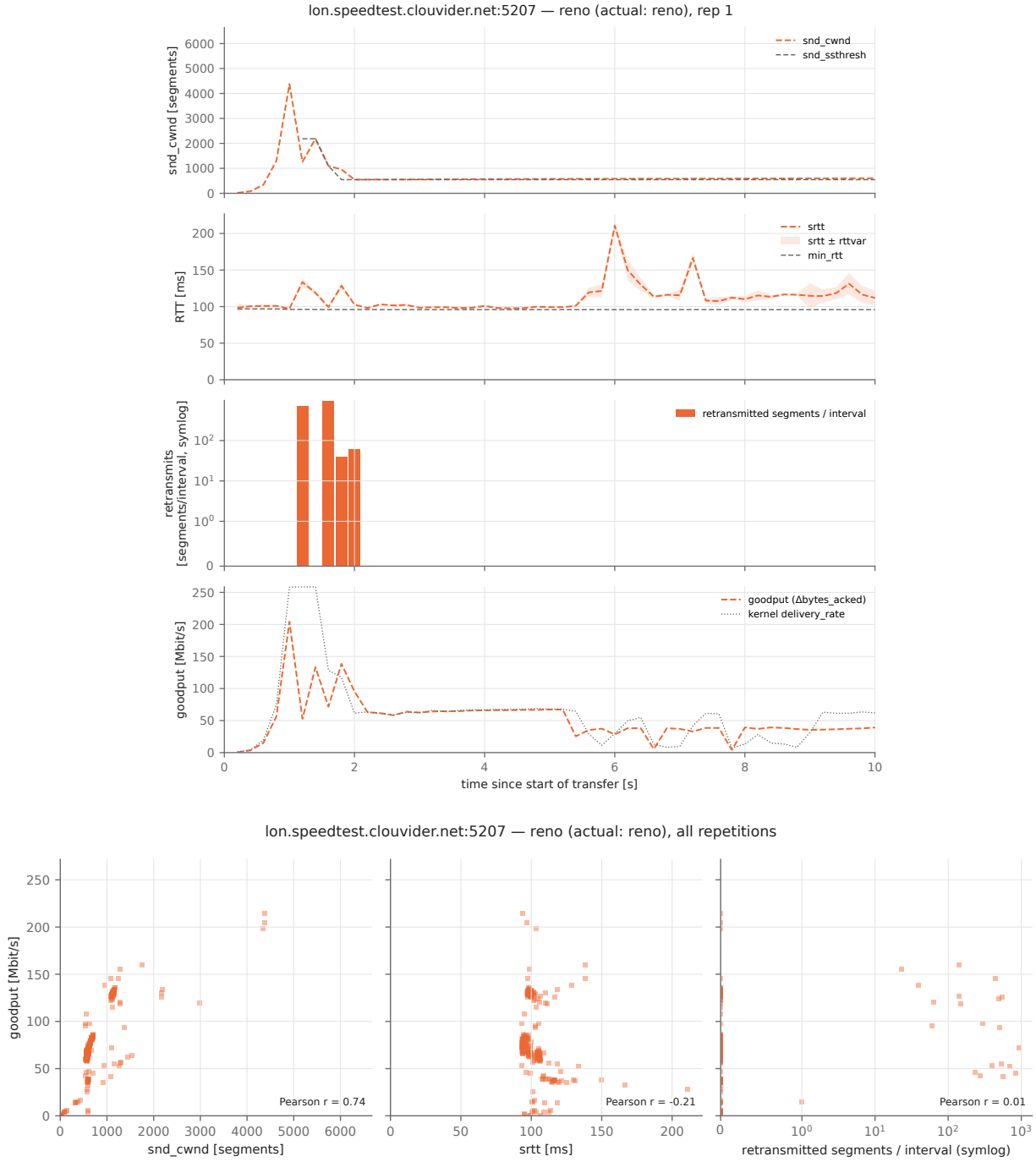


Figure 5: Reno (`reno`) at `lon.speedtest.clouvider.net:5207`: time series (top, repetition 1) and scatter (bottom, all repetitions). Files: `plots/q2_timeseries_reno.pdf`, `plots/q2_scatter_reno.pdf`.

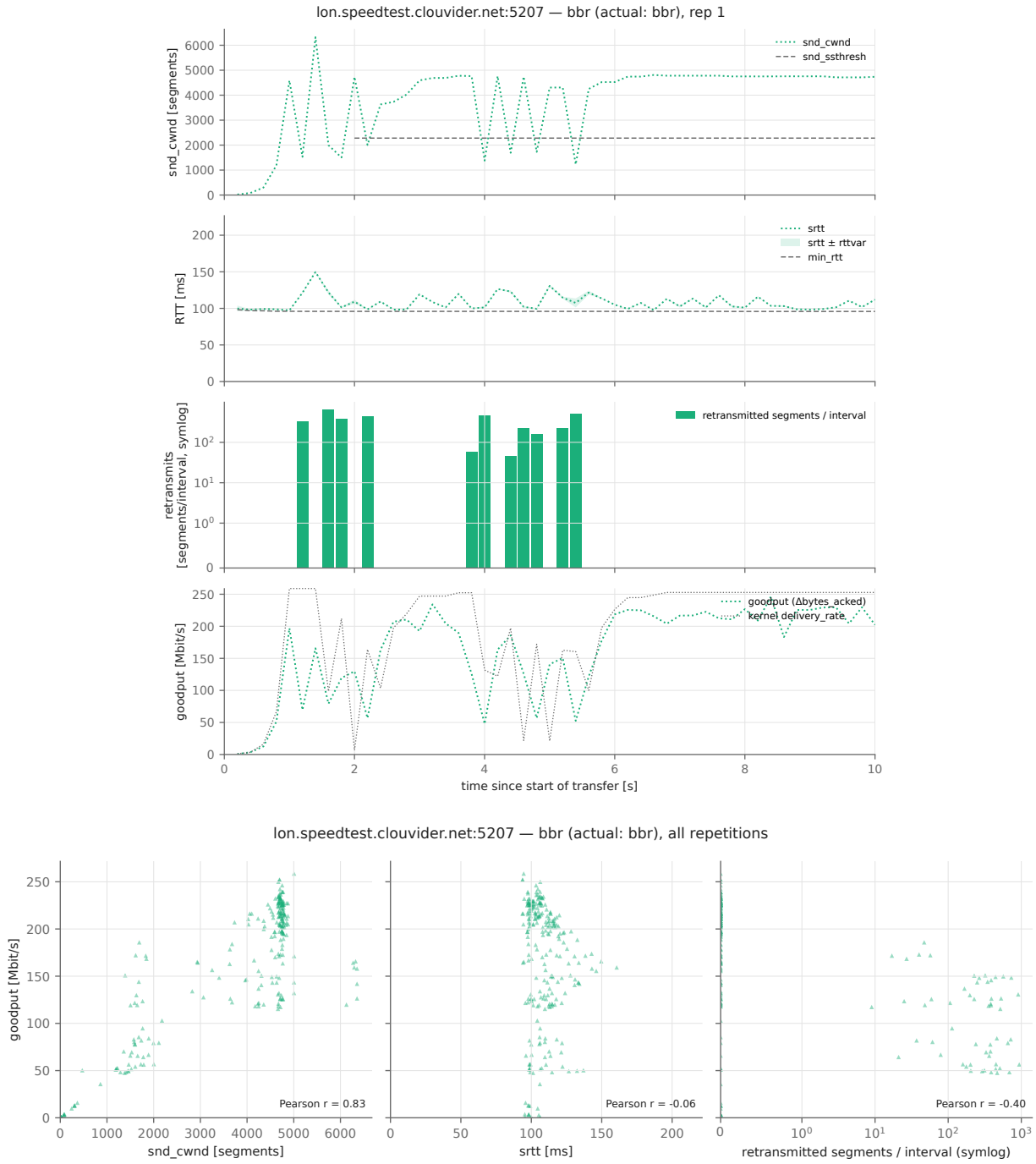


Figure 6: BBR (`bbr`) at `lon.speedtest.clouvider.net:5207`: time series (top, repetition 1) and scatter (bottom, all repetitions). Files: `plots/q2_timeseries_bbr.pdf`, `plots/q2_scatter_bbr.pdf`.

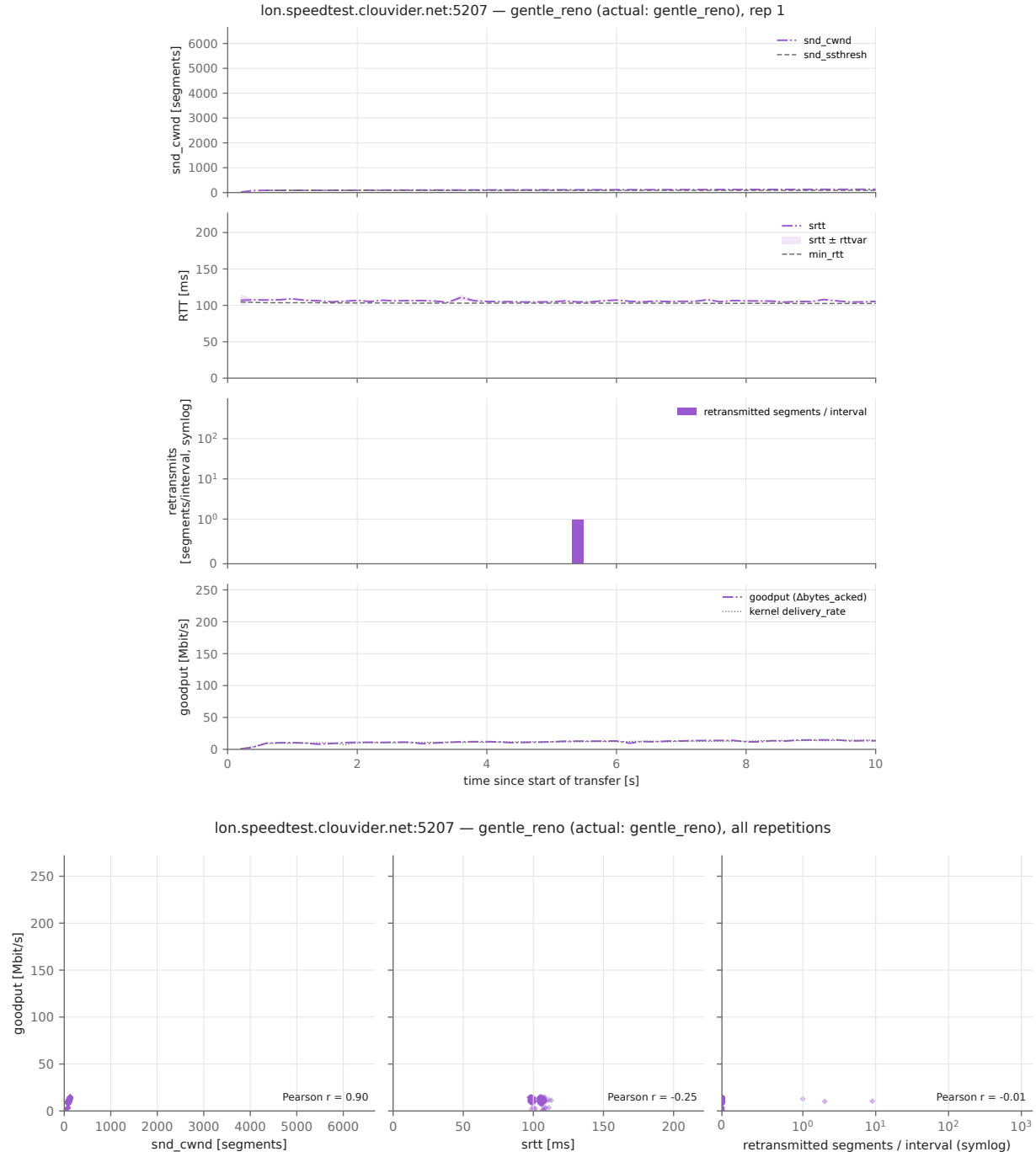


Figure 7: Gentle Reno (`gentle_reno`) at `lon.speedtest.clouvider.net:5207`: time series (top, repetition 1) and scatter (bottom, all repetitions). Files: `plots/q2_timeseries_gentle_reno.pdf`, `plots/q2_scatter_gentle_reno.pdf`.

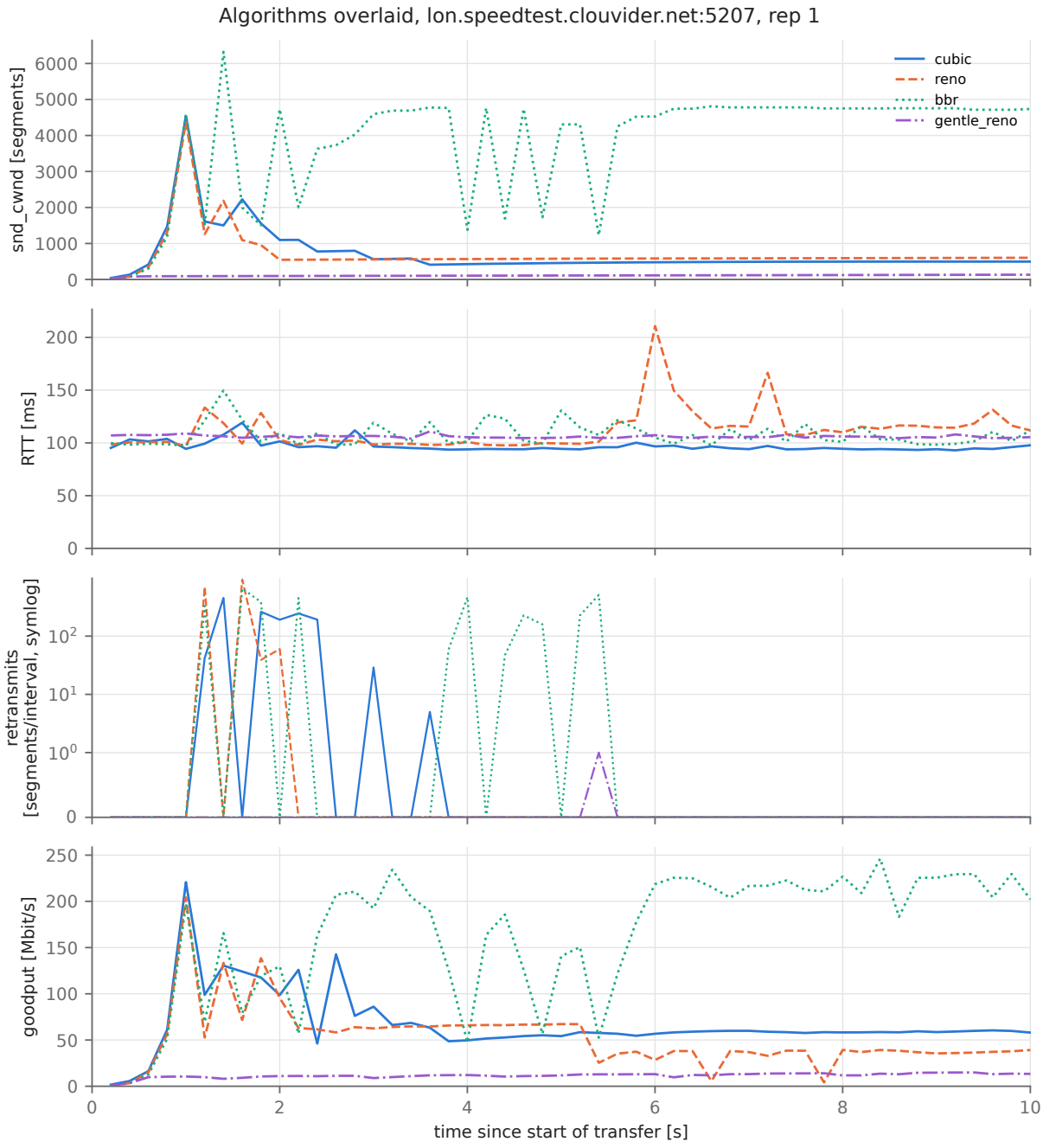


Figure 8: The four algorithms overlaid at `lon.speedtest.clouvider.net:5207`, repetition 1. File: `plots/q3_timeseries_overlay.pdf`.

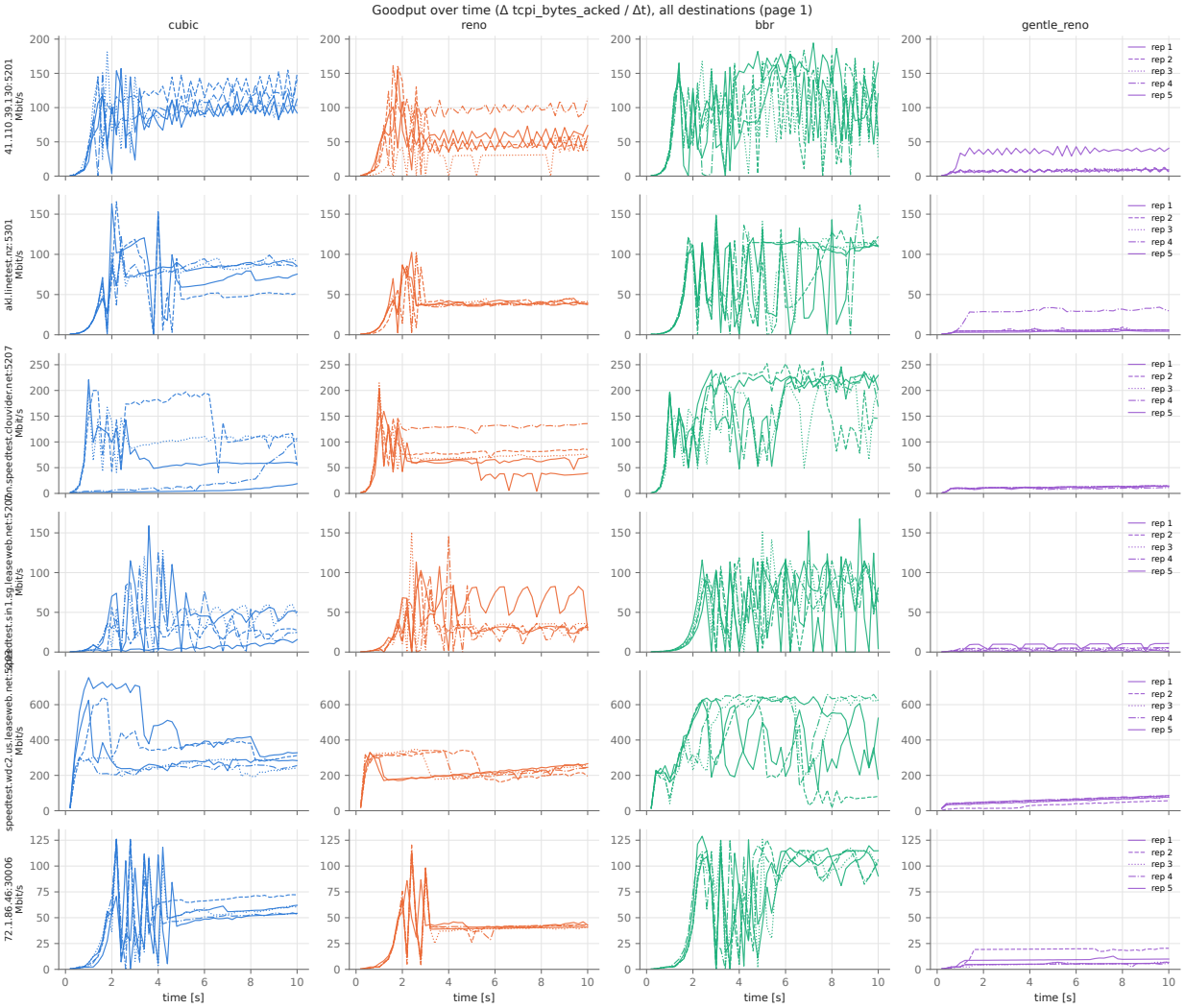


Figure 9: Goodput time series for every destination and algorithm (page 1 of 2). File: `plots/q1_goodput_timeseries.pdf`.

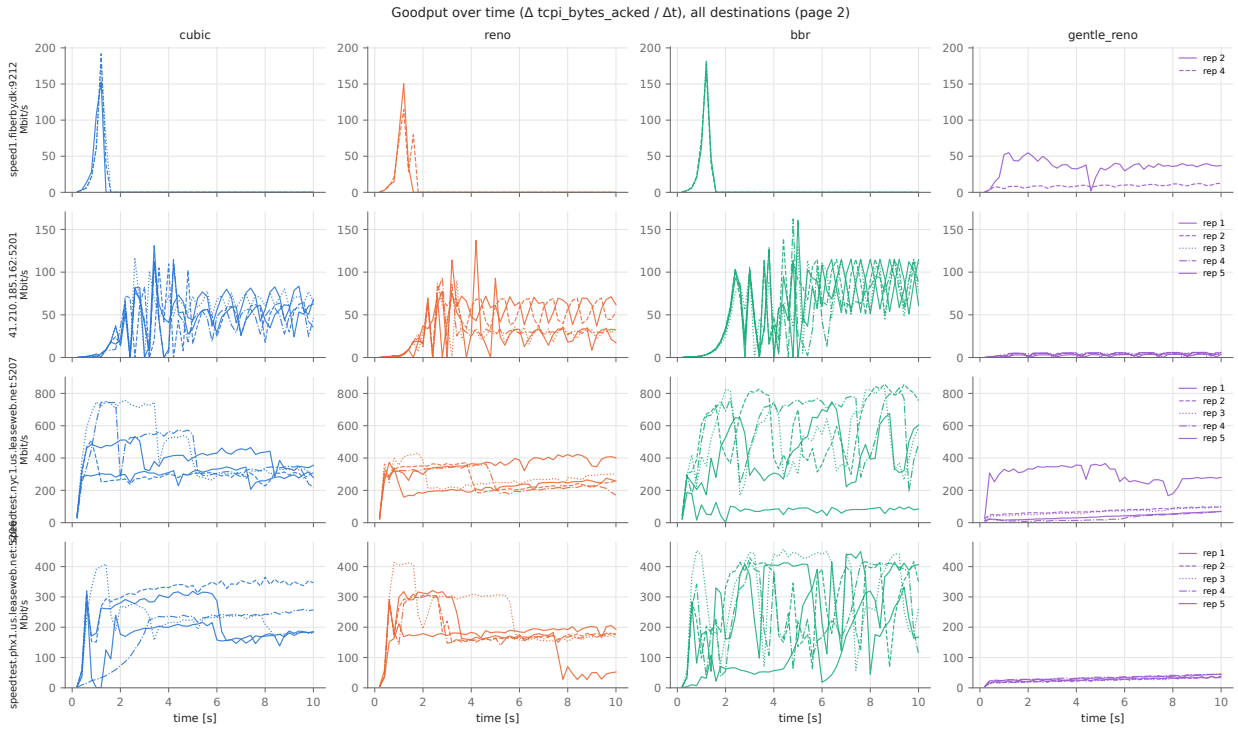


Figure 10: Goodput time series for every destination and algorithm (page 2 of 2).

B Test order

Execution order of all 200 tests (~~struck out~~ = failed). Each line runs the four algorithms back to back on one destination.

Rep	Dest	Destination	Algorithm order
1	1	`41.110.39.130:5201`	CUBIC → Reno → BBR → Gentle Reno
1	2	`akl.linetest.nz:5301`	Reno → BBR → Gentle Reno → CUBIC
1	3	`lon.speedtest.clouvider.net:5207`	BBR → Gentle Reno → CUBIC → Reno
1	4	`speedtest.sin1.sg.leaseweb.net:5207`	Gentle Reno → CUBIC → Reno → BBR
1	5	`speedtest.wdc2.us.leaseweb.net:5203`	CUBIC → Reno → BBR → Gentle Reno
1	6	`72.1.86.46:30006`	Reno → BBR → Gentle Reno → CUBIC
1	7	`speed1.fiberby.dk:9212`	BBR → ~~Gentle Reno~~ → CUBIC → ~~Reno~~
1	8	`41.210.185.162:5201`	Gentle Reno → CUBIC → Reno → BBR
1	9	`speedtest.nyc1.us.leaseweb.net:5207`	CUBIC → Reno → BBR → Gentle Reno
1	10	`speedtest.phx1.us.leaseweb.net:5206`	Reno → BBR → Gentle Reno → CUBIC
2	1	`41.110.39.130:5201`	Reno → BBR → Gentle Reno → CUBIC
2	2	`akl.linetest.nz:5301`	BBR → Gentle Reno → CUBIC → Reno
2	3	`lon.speedtest.clouvider.net:5207`	Gentle Reno → CUBIC → Reno → BBR
2	4	`speedtest.sin1.sg.leaseweb.net:5207`	CUBIC → Reno → BBR → Gentle Reno
2	5	`speedtest.wdc2.us.leaseweb.net:5203`	Reno → BBR → Gentle Reno → CUBIC
2	6	`72.1.86.46:30006`	BBR → Gentle Reno → CUBIC → Reno
2	7	`speed1.fiberby.dk:9212`	Gentle Reno → ~~CUBIC~~ → Reno → ~~BBR~~
2	8	`41.210.185.162:5201`	CUBIC → Reno → BBR → Gentle Reno
2	9	`speedtest.nyc1.us.leaseweb.net:5207`	Reno → BBR → Gentle Reno → CUBIC
2	10	`speedtest.phx1.us.leaseweb.net:5206`	BBR → Gentle Reno → CUBIC → Reno

Rep	Dest	Destination	Algorithm order
3	1	41.110.39.130:5201	BBR → Gentle Reno → CUBIC → Reno
3	2	akl.linetest.nz:5301	Gentle Reno → CUBIC → Reno → BBR
3	3	lon.speedtest.clouvider.net:5207	CUBIC → Reno → BBR → Gentle Reno
3	4	speedtest.sin1.sg.leaseweb.net:5207	Reno → BBR → Gentle Reno → CUBIC
3	5	speedtest.wdc2.us.leaseweb.net:5203	BBR → Gentle Reno → CUBIC → Reno
3	6	72.1.86.46:30006	Gentle Reno → CUBIC → Reno → BBR
3	7	speed1.fiberby.dk:9212	CUBIC → ~~Reno~~ → BBR → ~~Gentle Reno~~
3	8	41.210.185.162:5201	Reno → BBR → Gentle Reno → CUBIC
3	9	speedtest.nyc1.us.leaseweb.net:5207	BBR → Gentle Reno → CUBIC → Reno
3	10	speedtest.phx1.us.leaseweb.net:5206	Gentle Reno → CUBIC → Reno → BBR
4	1	41.110.39.130:5201	Gentle Reno → CUBIC → Reno → BBR
4	2	akl.linetest.nz:5301	CUBIC → Reno → BBR → Gentle Reno
4	3	lon.speedtest.clouvider.net:5207	Reno → BBR → Gentle Reno → CUBIC
4	4	speedtest.sin1.sg.leaseweb.net:5207	BBR → Gentle Reno → CUBIC → Reno
4	5	speedtest.wdc2.us.leaseweb.net:5203	Gentle Reno → CUBIC → Reno → BBR
4	6	72.1.86.46:30006	CUBIC → Reno → BBR → Gentle Reno
4	7	speed1.fiberby.dk:9212	Reno → ~~BBR~~ → Gentle Reno → ~~CUBIC~~
4	8	41.210.185.162:5201	BBR → Gentle Reno → CUBIC → Reno
4	9	speedtest.nyc1.us.leaseweb.net:5207	Gentle Reno → CUBIC → Reno → BBR
4	10	speedtest.phx1.us.leaseweb.net:5206	CUBIC → Reno → BBR → Gentle Reno
5	1	41.110.39.130:5201	CUBIC → Reno → BBR → Gentle Reno
5	2	akl.linetest.nz:5301	Reno → BBR → Gentle Reno → CUBIC
5	3	lon.speedtest.clouvider.net:5207	BBR → Gentle Reno → CUBIC → Reno
5	4	speedtest.sin1.sg.leaseweb.net:5207	Gentle Reno → CUBIC → Reno → BBR
5	5	speedtest.wdc2.us.leaseweb.net:5203	CUBIC → Reno → BBR → Gentle Reno
5	6	72.1.86.46:30006	Reno → BBR → Gentle Reno → CUBIC
5	7	speed1.fiberby.dk:9212	BBR → ~~Gentle Reno~~ → CUBIC → ~~Reno~~
5	8	41.210.185.162:5201	Gentle Reno → CUBIC → Reno → BBR
5	9	speedtest.nyc1.us.leaseweb.net:5207	CUBIC → Reno → BBR → Gentle Reno
5	10	speedtest.phx1.us.leaseweb.net:5206	Reno → BBR → Gentle Reno → CUBIC